

UT01 – Tecnologías de programación sobre clientes web

ACTIVIDAD EVALUABLE (RA1) – PLANTILLA DE ENTREGA

Nombre y Apellidos del
alumno/a:

[Jeronimo Hernandez Valoyes]

Fecha de entrega:

18/09/2026

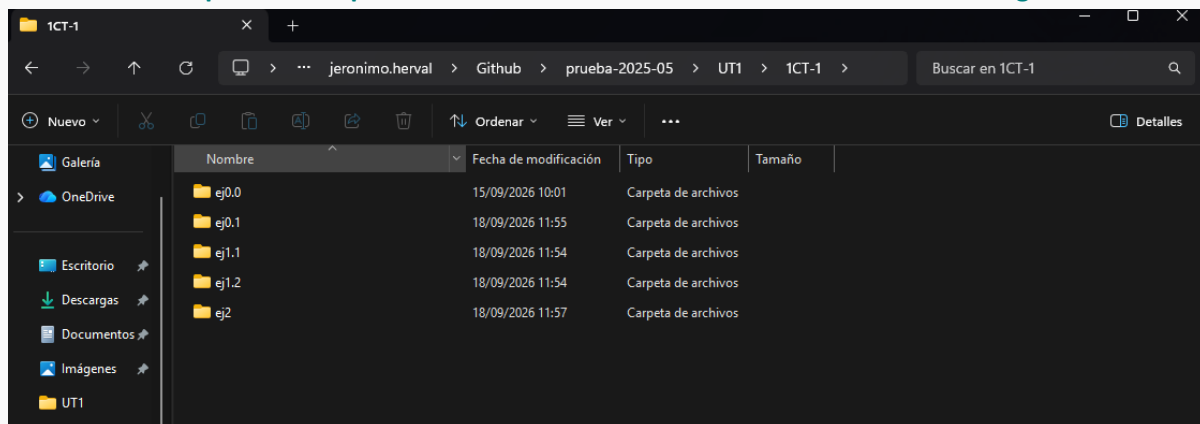
URL del repositorio GitHub:

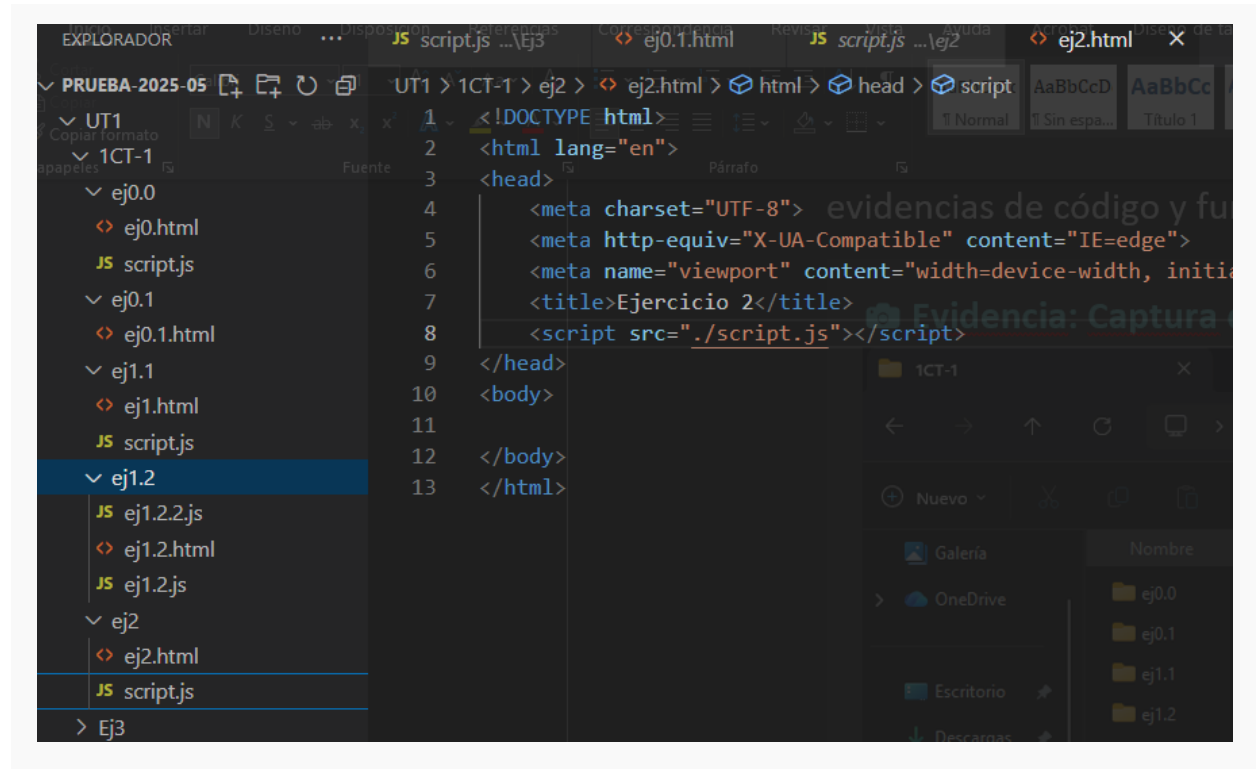
<https://github.com/JeronimoHernandezz/JS-2026.git>

1. Repositorio y Estructura de la Actividad (0,50 pts)

El repositorio de GitHub ha sido configurado bajo la estructura obligatoria **ACTIVIDADES ENTREGABLES / ACT-1 /** y compartido con la profesora (aralvarezr@educa.jcyl.es). A continuación se presentan las evidencias de código y funcionamiento de cada ejercicio.

 **Evidencia: Captura del explorador de archivos donde se vean todos los archivos generados.**





Ejercicio 1.1 – Carga Diferida con 'defer'

- Estructura: Archivo **index.html** con `<script defer src="..."></script>` ubicado en la cabecera `<head>`. El script genera los números del 1 al 100 en el cuerpo del documento mediante `document.body.innerHTML += "<p>"+i+"</p>"`.

Explica aquí qué ocurre si eliminas el atributo defer de la etiqueta `<script>`

Si quitamos el `defer` el script se ejecuta muy rapido cuando el body todavia no esta creado, por eso el `document.body` puede dar error como el ejemplo que vimos enc lase y los numeros del 1 al 100 no aparecerian correctamente, con el `defer` el script espera que el html termine de cargarse antes de ejecutarse, y asi el body ya estaria displonible y el programa funcionaria

Ejercicio 1.2 – Carga Asíncrona con 'async'

- Estructura: Archivo **index.html** con dos scripts asíncronos enlazados en el `<head>` mediante `<script async src="..."></script>`. Uno muestra los números del 1 al 100 y el otro del 900 al 1000.

📷 Explica aquí qué se pretende demostrar y si se ha conseguido con este ejemplo

```
[
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio 1.2</title>

  <script async src="./ej1.2.js"></script>
  <script async src = "./ej1.2.2.js"></script>
</head>
<body>
</body>
</html>
```

cc]

```
script.js  ej1.1.html  ej2.html  ej1.2.2.js
1 > 1CT-1 > ej1.2 > JS ej1.2.2.js > ...
1  const MAX2 = 1000;
2  let seguir2 = true;
3  for (let l = 900; seguir2 == true; l++) {
4    document.body.innerHTML += "<p>" + l + "</p>";
5    if (l == MAX2) seguir2 = false;
6  }
7
```

```
1 > 1CT-1 > ej1.2 > JS ej1.2.js > ...
1  const MAX = 100;
2  let seguir = true;
3  for (let i = 1; seguir == true; i++) {
4    document.body.innerHTML += "<p>" + i + "</p>";
5    if (i == MAX) seguir = false;
6  }
7
8
```

Con este ejemplo se busca utilizar la forma asincrónica al vincular el HTML con los JS de manera independiente y cada uno se ejecuta en cuanto se carguen sin tener que esperar que el primero acabe, por eso el orden puede variar, puede cargar primero el script de los números del 1 al 100 o primero el script de los números de 900 al 1000 pero eso es otra manera de comprobar el funcionamiento del `async`

2. Depuración en Visual Studio Code (0,75 pts)

La sesión de depuración se realiza sobre el archivo JavaScript ubicado en la carpeta **Ej.2**. A continuación se documentan los elementos clave de la interfaz de depuración de VS Code.

2.1. Punto de Ruptura (Breakpoint)

Captura de Pantalla: Punto de ruptura (Breakpoint) activo en el código del Ej.2



Añadimos el punto de ruptura e iniciamos la depuración]

2.2. Botones de Control de Depuración de VS Code

Identificación y explicación técnica del efecto de los 6 botones del panel flotante de depuración:

Botón / Acción	Icono	Atajo de Teclado	Efecto / Descripción Técnica
Continuar (Continue)		F5	Continúa la ejecución del script
Paso a paso por encima (Step Over)		F10	Ejecuta la siguiente línea del código del script
Paso a paso por dentro (Step Into)		F11	Salta a la siguiente función del script

Paso a paso para salir
(Step Out)



MAYUS + F11

Continúa la ejecución hasta salir del script

Reiniciar (Restart)



CTRL + MAYUS +
F5

Reinicia la ejecución del script

Detener (Stop)

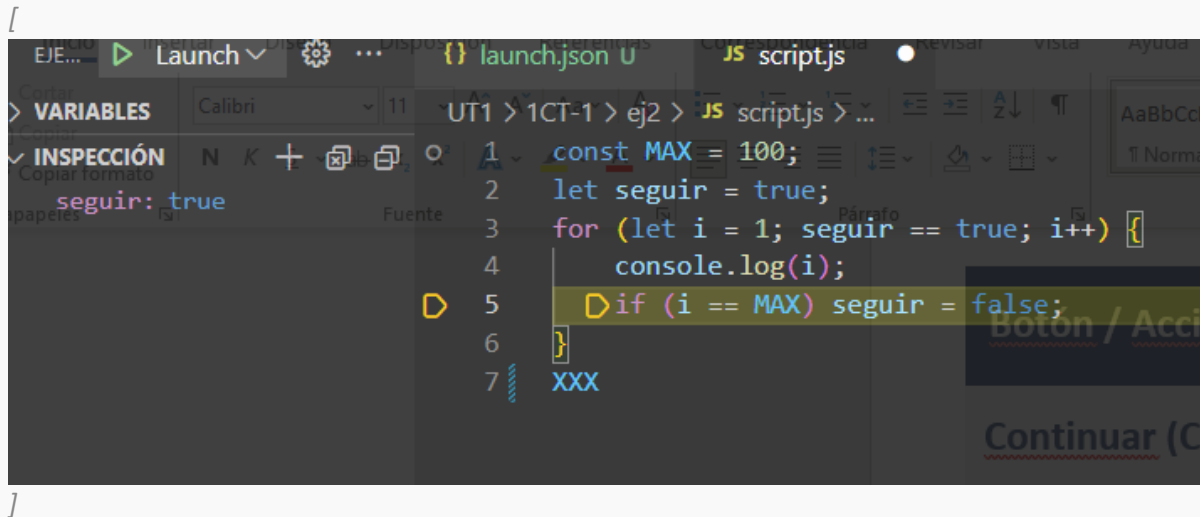


MAYUS + F5

Finaliza la depuración del script

2.3. Inspección de Código: Zonas Watch y Variables

 Captura de Pantalla: Panel 'Watch' con expresiones en seguimiento y 'Variables' en un estado concreto



3. Ejecución con Node.js (0,25 pts)

Ejecución del script de la carpeta Ej2 en el entorno de servidor local Node.js mediante la terminal integrada de Visual Studio Code.

📸 Captura de Pantalla: Terminal integrada con comando 'node -v', la ruta completa y la ejecución del script mostrando la salida personalizada

```
[
  PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\jeronimo.herval\Github\prueba-2025-05> node -v
v18.17.1
PS C:\Users\jeronimo.herval\Github\prueba-2025-05> ]
```

Nos enseña la version de node js que estamos usando actualmente.

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\jeronimo.herval\Github\prueba-2025-05> cd '..\UT1\1CT-1\Ej2 - Personal\'
PS C:\Users\jeronimo.herval\Github\prueba-2025-05\UT1\1CT-1\Ej2 - Personal> ]
```

La ruta complete del ej2 Personal

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\jeronimo.herval\Github\prueba-2025-05> cd '..\UT1\1CT-1\Ej2 - Personal\'
PS C:\Users\jeronimo.herval\Github\prueba-2025-05\UT1\1CT-1\Ej2 - Personal> node .\script.js
1 Personal
2 Personal
3 Personal
4 Personal
5 Personal
6 Personal
7 Personal
8 Personal
9 Personal
10 Personal
11 Personal
12 Personal
13 Personal
14 Personal
```

4. Lista de Cotejo / Autoevaluación

Calificación máxima: 1,50 pts · Escala: Sí = puntuación total · Parcial = 50 % · No = 0

Criterio de Evaluación	Puntuación Máx.	Sí/Parcial /No
1. Repositorio y estructura	0,50	SI
· Repositorio compartido con aralvarezr@educa.jcyl.es y accesible (sin 404).	0,05	SI
· Carpeta ACTIVIDADES ENTREGABLES / ACT-1 con el documento y ejercicios.	0,05	SI
· Ej.0.0: .html con <script src="..."> y script.js que muestra 'Hola mundo'.	0,10	SI
· Ej.0.1: .html con script en línea (sin src) y resultado coincidente con Ej.0.0.	0,10	SI
· Ej.1.1: Script externo con defer en <head> escribe 1-100 en document.body.	0,10	SI
· Ej.1.2: Dos scripts con async en <head>, uno 1-100 y otro 900-1000.	0,10	SI
2. Depuración en Visual Studio Code	0,75	SI
· Puntos de ruptura (breakpoints) con ejecución pausada y línea resaltada.	0,20	SI
· 6 Botones de avance identificados y explicados con sus efectos.	0,20	SI
· Zona Watch con expresión añadida y valor durante la ejecución.	0,15	SI
· Zona Variables comentando cambios de valor en ejecución.	0,15	NO
· Calidad de evidencia: capturas legibles.	0,05	SI
3. Ejecución con Node.js	0,25	SI
· Ejecución del commando en la ruta correcta.	0,10	SI
· Salida por consola correcta, incluyendo el nombre del alumno	0,10	SI
· Entorno visible: terminal integrada de VS Code y versión de Node (node -v).	0,05	SI
CALIFICACIÓN OBTENIDA		1,35

Observaciones:

[Si quieres añadir alguna observación sobre esta actividad, puedes hacerlo aquí]